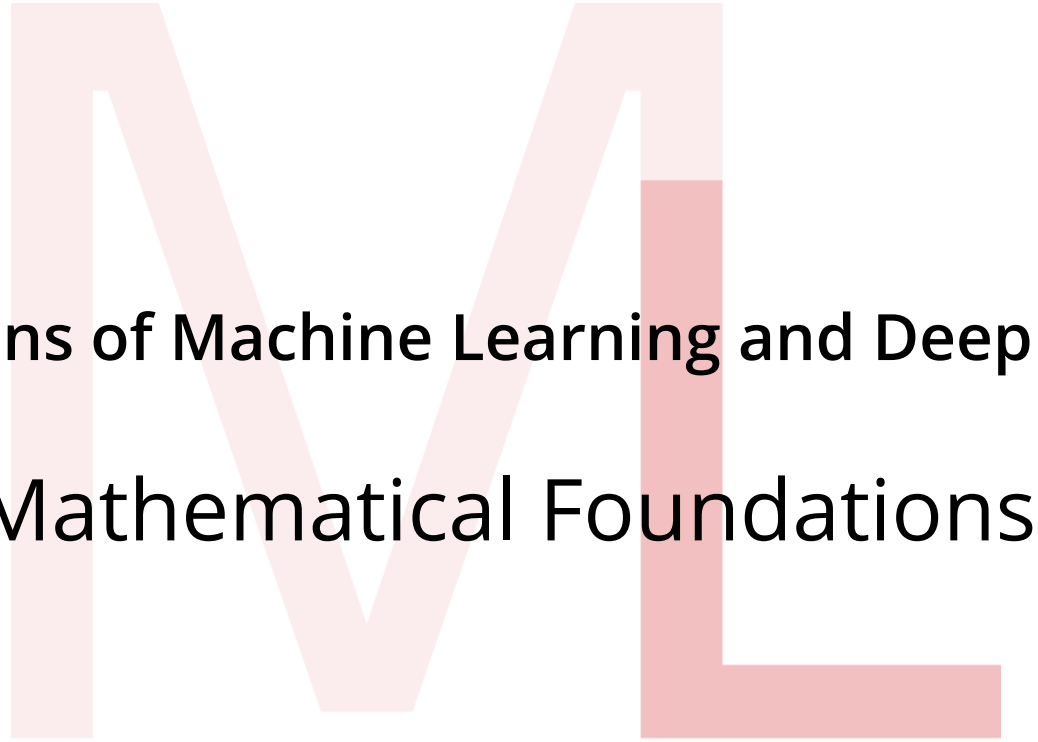


Foundations of Machine Learning and Deep Learning

Mathematical Foundations

University Master's Degree in
Machine Learning

upna

Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Linear algebra and pytorch tensors

- You are familiar with the tools of linear algebra. Review in the context of deep learning
  - How data is structured in tensors
  - How matrix multiplication is fundamental in a neural network layer
  - How the size of vectors, measured by norms, is used to prevent overfitting
  - How eigendecomposition provides a deeper insight into data

Scalars

- A scalar is a single number
- Denoted by lowercase letters: x, y, z
- All real-valued scalars: $\mathbb{R}$

$$x, y, z \in \mathbb{R}$$

- **In Deep Learning**
  - A single feature or data point
  - The age of a house
  - The number of rooms
  - The humidity in a place
  - The heigh of a person

Vectors

- An ordered array of numbers of fixed length
  - A point in an n-dimensional space
  - An arrow from the origin to that point (direction and magnitude)
- Denoted by bold lowercase letters: $\mathbf{x}, \mathbf{y}, \mathbf{z}$
  - For elements we use a subscript (no bold): $x_1, x_2, \ldots, x_n$
  - Dimensionality: $\boldsymbol{x} \in \mathbb{R}^n$ (the vector has n elements)
- **In Machine/Deep Learning**
  - A single data instance or a feature vector
    - Tabular data: patient1 = [age, blood_pressure, colesterol_level, weight]
    - Natural language processing: a word represented by an embedding
    - Images: a greyscale image flattened into a single vector

Matrices

- A 2D array of numbers
  - A collection of vectors: a Dataset
  - A linear transformation: function to transform vectors from one space to another
- Denoted by bold capital letters: $\mathbf{A}, \mathbf{B}, \mathbf{C}$
  - Dimensionality: $\boldsymbol{A} \in \mathbb{R}^{n \times m}$ (the vector has n rows and m columns)
  - For elements we use two subscripts (row and column): $a_{11}, a_{12}, \ldots, a_{mn}$

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

- **In Deep Learning**
  - Weights in a dense (fully connected) layer

Tensors

- Generalization of this concept to an arbitrary number of dimensions:
  - 0D tensor: a scalar
  - 1D tensor: a vector
  - 2D tensor: a matrix
  - 3D tensor: a n-dimensional array
- Tensors are the default data structure in all modern Deep Learning frameworks (PyTorch, TensorFlow, JAX)

- Examples
  - Color image data → 3D tensor
  - Batch of color images → 4D tensor

Matrix multiplication

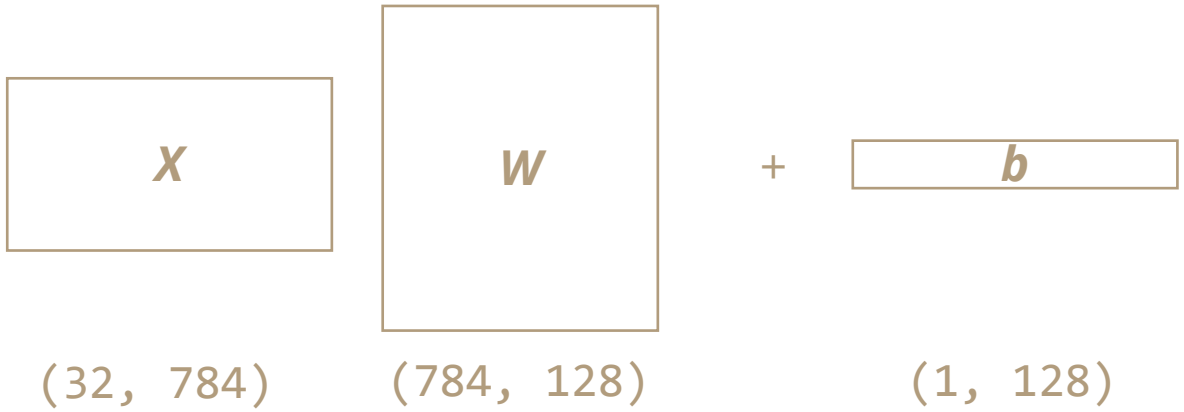
- $\boldsymbol{A} \in \mathbb{R}^{n \times k}$, $\mathbf{B} \in \mathbb{R}^{k \times m}$. $\boldsymbol{C} = \boldsymbol{AB}$ $(\boldsymbol{C} = \boldsymbol{A} @ \boldsymbol{B}) \in \mathbb{R}^{n \times m}$. Inner dimensions must match
  - $\boldsymbol{A} = \begin{bmatrix} a_{11} & \cdots & a_{1k} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nk} \end{bmatrix}$, $\boldsymbol{B} = \begin{bmatrix} b_{11} & \cdots & b_{1m} \\ \vdots & \ddots & \vdots \\ b_{k1} & \cdots & b_{km} \end{bmatrix}$
  - $\boldsymbol{A} = \begin{bmatrix} \boldsymbol{a}_1^T \\ \vdots \\ \boldsymbol{a}_n^T \end{bmatrix}$, $\boldsymbol{B} = [\boldsymbol{b}_1 \quad \ldots \quad \boldsymbol{b}_m]$

    $\boldsymbol{a}_i^T \in \mathbb{R}^k$ $\quad$ i^{th} row of matrix $\boldsymbol{A}$

    $\boldsymbol{b}_j \in \mathbb{R}^k$ $\quad$ j^{th}column of matrix $\boldsymbol{B}$
  - $C = AB = \begin{bmatrix} \boldsymbol{a}_1^T \\ \vdots \\ \boldsymbol{a}_n^T \end{bmatrix} [\boldsymbol{b}_1 \quad \ldots \quad \boldsymbol{b}_m] = \begin{bmatrix} \boldsymbol{a}_1^T \boldsymbol{b}_1 & \cdots & \boldsymbol{a}_1^T \boldsymbol{b}_m \\ \vdots & \ddots & \vdots \\ \boldsymbol{a}_n^T \boldsymbol{b}_1 & \cdots & \boldsymbol{a}_n^T \boldsymbol{b}_m \end{bmatrix}$

Matrix multiplication

- In Deep Learning
  - Linear layer: $\boldsymbol{Z} = \boldsymbol{XW} + \boldsymbol{b}$
    - $\boldsymbol{X}$ input data matrix (`batch_size, in_features`)
    - $\boldsymbol{W}$ weight matrix (`in_features, num_neurons`)
    - $\boldsymbol{XW}$ (`batch_size, num_neurons`)
    - $\boldsymbol{b}$ bias (`1, num_neurons`)



Other tensor operations

- Applied independently to each element of a tensor
  - $\boldsymbol{A} + \boldsymbol{B}$
  - $\boldsymbol{A} - \boldsymbol{B}$
  - $\boldsymbol{A} \odot \boldsymbol{B}$ (Hadamard product)
  - Sum of elements of a matrix
  - Dot product (inner product of two vectors - $\langle \boldsymbol{x}, \boldsymbol{y} \rangle$)
  - ...

Norms

- A norm is a function that maps a vector to a non-negative value, representing its length or magnitude. A norm ||·|| must satisfy:
- $\left||\boldsymbol{x}|\right| \geq 0$ (non-negative)
- $||\boldsymbol{x}||$ =0 if and only if $\boldsymbol{x} = 0$
- $\left||\alpha\boldsymbol{x}|\right| = |\alpha|\ ||\boldsymbol{x}||$ (scales with constants)
- $\left||\boldsymbol{x} + \boldsymbol{y}|\right| \leq \left||\boldsymbol{x}|\right| + ||\boldsymbol{y}||$ (triangle inequality)

Norms

- ℓ_2 norm = Euclidean norm
  - $\left|\left|\boldsymbol{x}\right|\right|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$
- ℓ_1 norm = Manhattan distance
  - $\left|\left|\boldsymbol{x}\right|\right|_1 = \sum_{i=1}^{n} |x_i|$
- ℓ_p norm (general case)
  - $\left|\left|\boldsymbol{x}\right|\right|_p = \sqrt[p]{\sum_{i=1}^{n} |x_i|^p}$
- For matrices – Frobenius norm (similar to ℓ_2)
  - $\left|\left|X\right|\right|_F = \sqrt{\sum_{i=1}^{n} \sum_{j=1}^{m} x_{ij}^2}$

Eigendecomposition

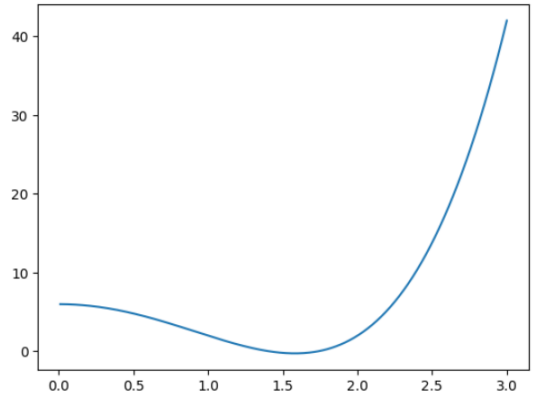
- When applying a matrix to a vector, the hyperplane will be stretched, sheared and rotated.
  - There are some vectors that remain on their original line through the origin → eigenvectors
  - Eigenvalues: the factor they are stretched ($\lambda > 1$), shrunk ($0 < \lambda < 1$) or flipped ($\lambda < 0$)

$$\boldsymbol{Av} = \lambda \boldsymbol{v}$$

- **In Deep Learning**
  - The eigendecomposition of the Hessian matrix give insights about the curvature of the los function. Inspiration for some optimization algorithms

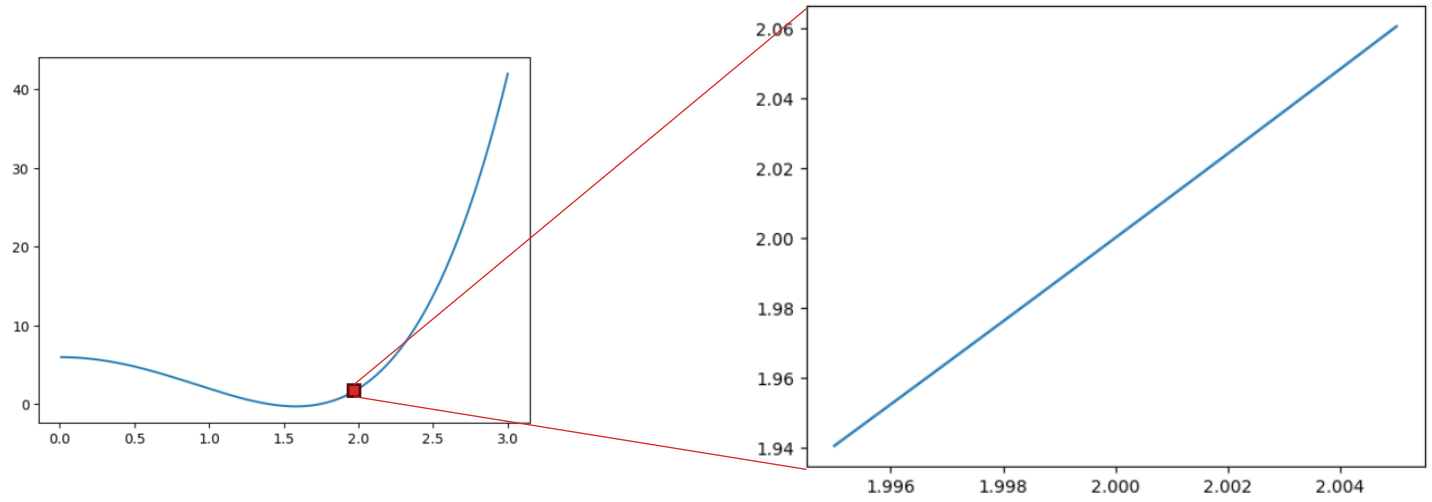
Differentiation

- Derivative: rate of change in a function with respect to changes in its arguments
- $f: \mathbb{R} \to \mathbb{R} \quad f'(x) = \lim_{h \to 0} \frac{f(x+h)-f(x)}{h}$
- Example: $f(x) = x^4 - 5x^2 + 6$



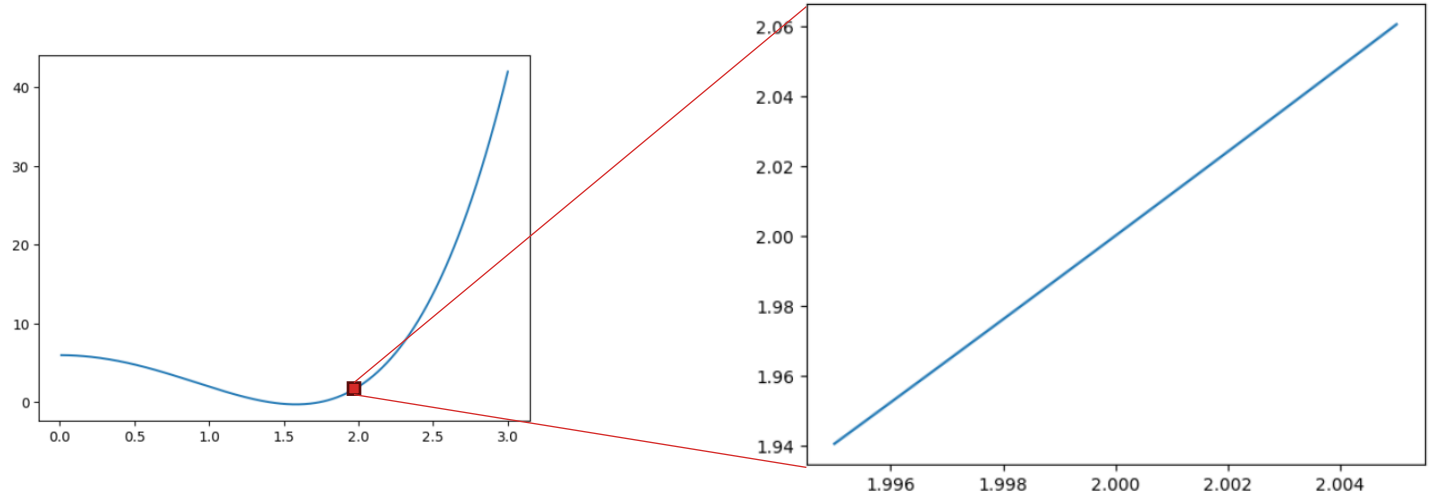
Differentiation

- Derivative: rate of change in a function with respect to changes in its arguments
- $f: \mathbb{R} \to \mathbb{R} \quad f'(x) = \lim_{h \to 0} \frac{f(x+h)-f(x)}{h}$
- Example: $f(x) = x^4 - 5x^2 + 6$



Differentiation

- Derivative: rate of change in a function with respect to changes in its arguments
- $f: \mathbb{R} \to \mathbb{R} \quad f'(x) = \lim_{h \to 0} \frac{f(x+h)-f(x)}{h}$
- Example: $f(x) = x^4 - 5x^2 + 6$

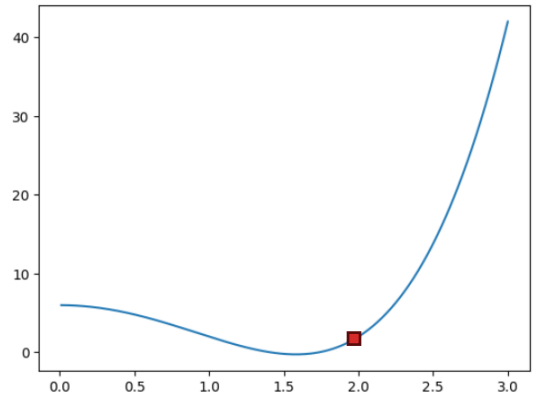


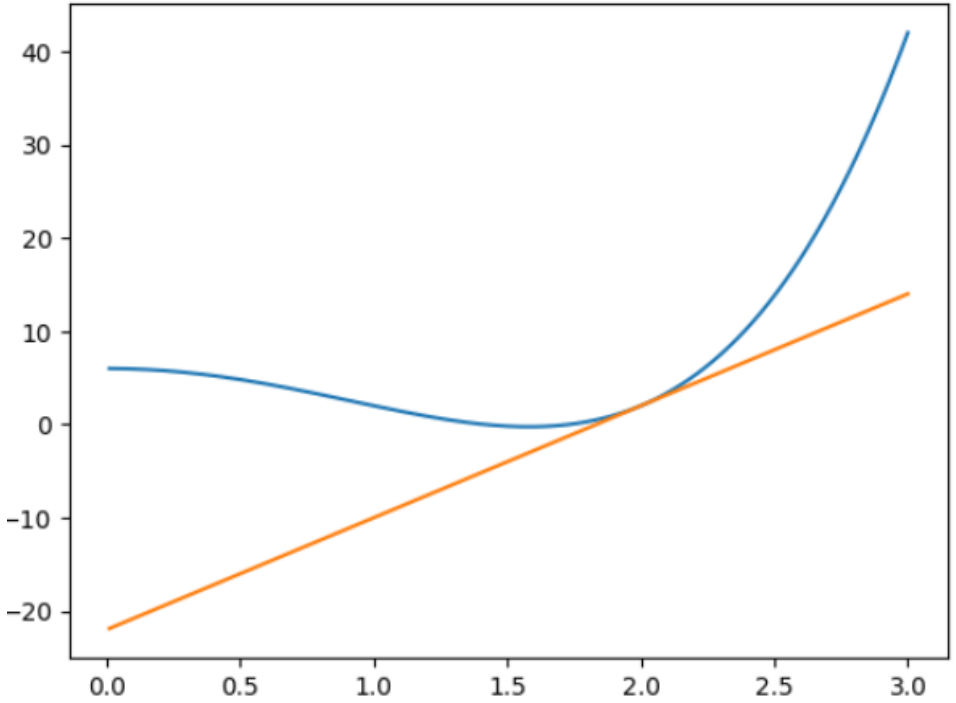

```
for eps in [0.1, 0.001, 0.0001, 0.00001]:
    print(f'eps={eps:.5f} -> {(f(torch.tensor(2+eps))\
    -f(torch.tensor(2)))/eps:.5f}')
```

```
eps=0.10000 -> 13.98100
eps=0.00100 -> 12.01820
eps=0.00010 -> 11.97815
eps=0.00001 -> 12.01630
```

Differentiation

- Derivative: rate of change in a function with respect to changes in its arguments
- $f: \mathbb{R} \to \mathbb{R} \quad f'(x) = \lim_{h \to 0} \frac{f(x+h)-f(x)}{h}$
- Example: $f(x) = x^4 - 5x^2 + 6$





Differentiation

- Common derivatives
  - Derivative of constants: $\frac{d}{dx}c = 0$
  - Derivative of linear functions: $\frac{d}{dx}(ax) = a$
  - Power rule: $\frac{d}{dx}x^n = nx^{n-1}$
  - Derivative of exponentials: $\frac{d}{dx}e^x = e^x$
  - Derivative of the logarithm: $\frac{d}{dx}log(x) = \frac{1}{x}$

Differentiation

- Derivative rules
  - Sum rule

$$\frac{d}{dx}\big(g(x)+h(x)\big)=\frac{dg}{dx}(x)+\frac{dh}{dx}(x)$$

  - Product rule

$$\frac{d}{dx}\big(g(x)\cdot h(x)\big)=g(x)\frac{dh}{dx}(x)+\frac{dg}{dx}(x)h(x)$$

  - Chain rule

$$\frac{d}{dx}g\big(h(x)\big)=\frac{dg}{dh}\big(h(x)\big)\cdot\frac{dh}{dx}(x)$$

Partial derivates

- Multivariate functions: $f(x_1, x_2, \dots, x_n)$
- Partial derivative with respect to the ith parameter

$$\frac{\partial y}{\partial x_i} = \lim_{h \to 0} \frac{f(x_1, \dots, x_{i-1}, x_i + h, x_{i+1}, \dots, x_n) - \mathrm{f}(\mathrm{x}_1, \dots, x_i, \dots, x_n)}{h}.$$

- Can be calculated treating the remaining parameters as constants

- The concatenation of partial derivatives of a multivariate function with respect to all its variables is called the **gradient** of the function → $\nabla f(x)$

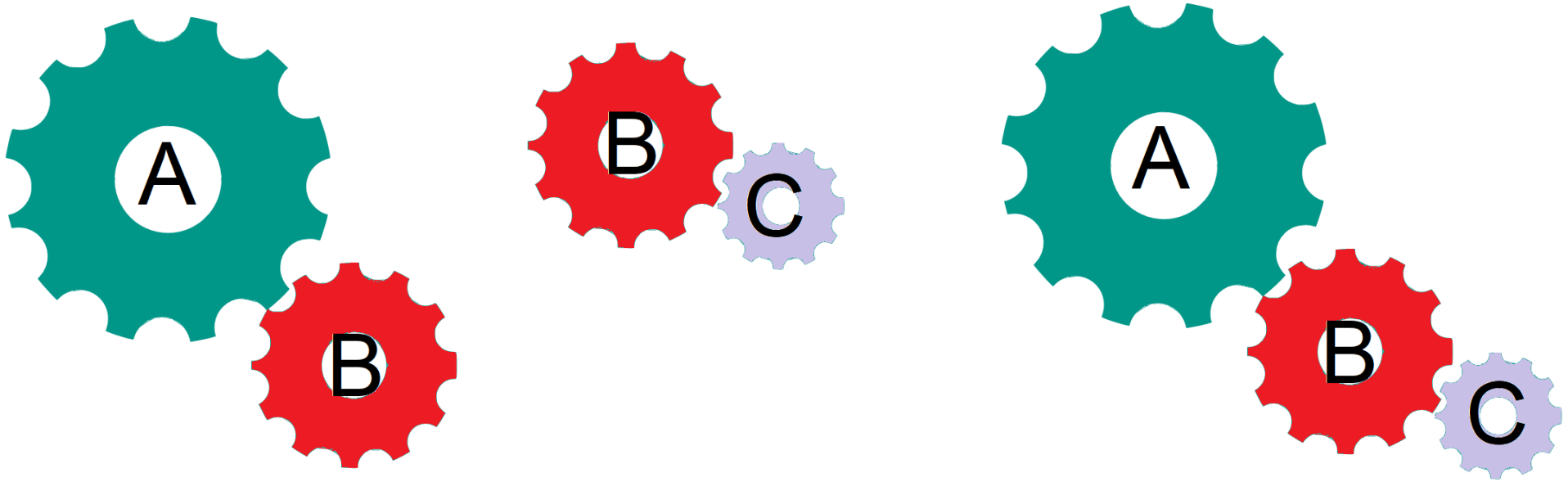
Partial derivatives

- **Example**
  - $f(a,b,c) = a^4b^3 - b^2c + a^2c^3 - c$
  - $\frac{\partial f}{\partial a} = 4a^3b^3 + 2ac^3$
  - $\frac{\partial f}{\partial b} = 3b^2a^4 - 2bc$
  - $\frac{\partial f}{\partial c} = -b^2 + 3c^2a^2 - 1$

  - $\nabla f = [4a^3b^3 + 2ac^3, 3b^2a^4 - 2bc, -b^2 + 3c^2a^2 - 1]$

Chain rule

- $\frac{d}{dx} g\big(h(x)\big) = \frac{dg}{dh}\big(h(x)\big) \cdot \frac{dh}{dx}(x)$



$g'(x) = 2x$ A small change in A (one turn of the gear) results in a change of 2 turns in B
$h'(x) = 2x$ A small change in B (one turn of the gear) results in a change of 2 turns in C
$g'(h'(x) = 4x$ A small change in A (one turn of the gear) results in a change of 4 turns in C

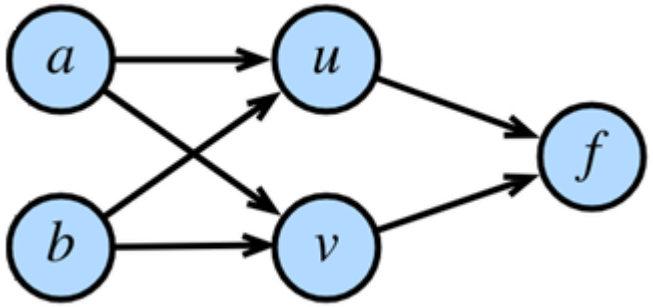
Multivariate chain rule

- When the change in one variable affects the final result through multiple paths
  - Sum all of them

- **Example**
  - If f depends on u and v, and u and v depend on a
  - $\frac{\partial f}{\partial a} = \frac{\partial f}{\partial u} * \frac{\partial u}{\partial a} + \frac{\partial f}{\partial v} * \frac{\partial v}{\partial a}$

Multivariate chain rule

- $f(u, v) = (u + v)^2$
- $u(a, b) = (a + b)^2$
- $v(a, b) = (a - b)^2$



- Calculate $\frac{\partial f}{\partial a}$ and $\frac{\partial f}{\partial b}$ using the chain rule and directly

Multivariate chain rule

- **Modularity:** It allows us to decompose a very complex derivative (∂f/∂w) into a series of very simple local derivatives (∂f/∂u, ∂u/∂a, ∂a/∂w, etc.).
- **Computational Efficiency:** By going backwards, we can reuse calculations. The gradient ∂f/∂u is computed once and then used to calculate ∂f/∂a and ∂f/∂b. This avoids recalculating things over and over again.